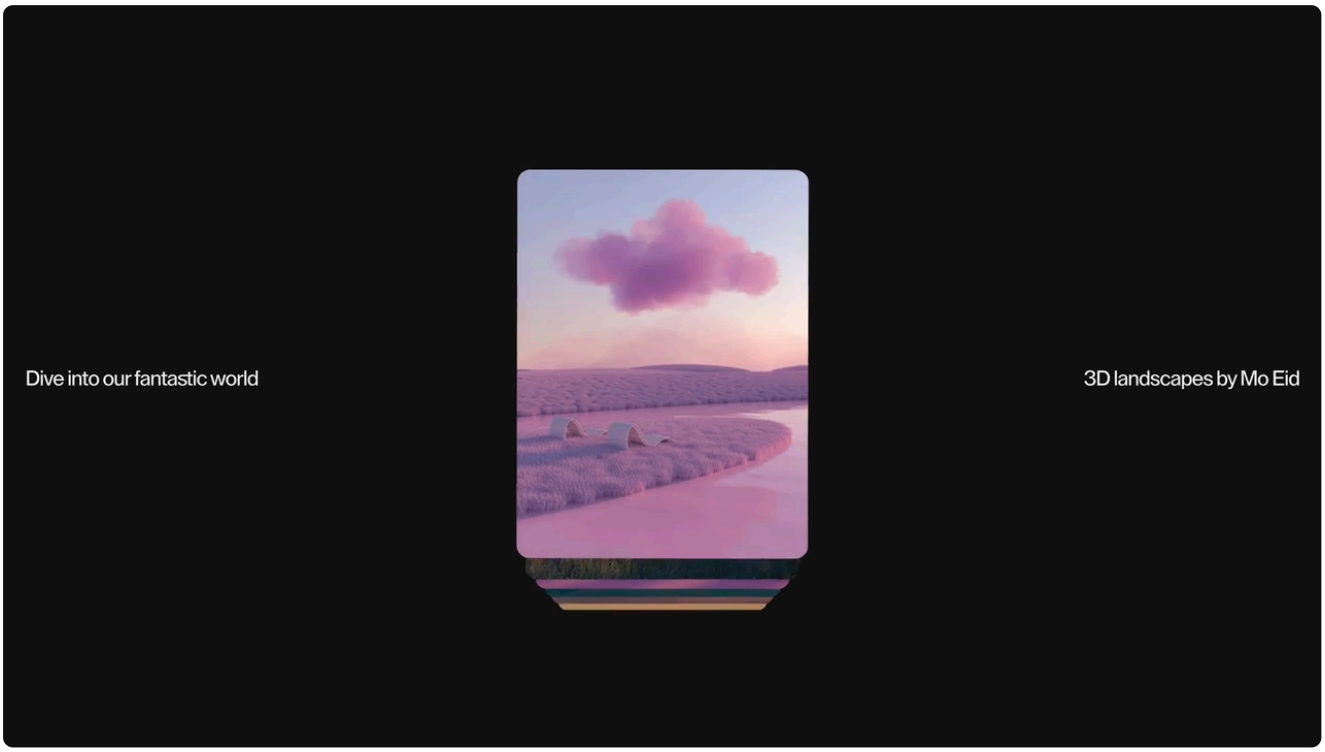
Made With Gsap

042
Smooth stacking images



In this effect, we will reveal a series of cards one by one. As soon as one card disappears, the remaining deck moves up towards us. The progression will happen as the user scrolls. Shall we take a look at it? Let's go!

HTML Structure

The structure is relatively simple here. The `container` division will remain pinned on the full height of the `pin-height` division. The images will be children of the `medias` division.

```
<div class="pin-height">
    <div class="container">
        <div class="medias">
            <img class="media" src="./assets/medias/1.png" alt="">
            <img class="media" src="./assets/medias/2.png" alt="">
            <img class="media" src="./assets/medias/3.png" alt="">
            ...
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS
Made With Gsap

Some CSS

Our `pin-height` division will set the height during which our `container` will remain pinned while scrolling. I chose a value of `500vh` for the scrolling distance so that my effect plays out completely. If I reduce this value, the images will scroll faster since there will be less distance to cover.

```
.mwg_effect042 .pin-height {
    height: 500vh;
}
```

Since we are going to move the cards on the Z axis, I activate CSS 3D for the `medias` division using the `perspective` property. I like to specify a perspective in `vw` to maintain the same result regardless of the screen width. The smaller the value, the more pronounced and deep the perspective will be.

```
.mwg_effect042 .medias {
    width: 22%;
    aspect-ratio: 0.75;
    perspective: 25vw;
}
```

Pinning the container

On the JavaScript side, I start by pinning the `container` division to the full height of `pin-height` using an instance of `ScrollTrigger.create()`. This instance also allows me to specify when to pin the container.

```
const root = document.querySelector('.mwg_effect042')
const pinHeight = root.querySelector('.pin-height')
const container = root.querySelector('.container')

ScrollTrigger.create({
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

```
    pin: container // the pinned section
})
```

To explain the code above: when the `pin-height` division reaches the top of the viewport, we start pinning the `container` until `pin-height` passes the bottom of the viewport. Thus, the `container` will remain perfectly visible on the screen over that distance.

Card placement

I choose a pixel value that will define the space between two cards on the Y and Z axes. This value will be stored in the `gap` variable. I then use the `gsap.set()` method to place the cards at the back of the deck. This method immediately sets properties of the target accordingly.

```
let gap = 30
const medias = root.querySelectorAll('.media')

gsap.set(medias, {
    y: gap * (medias.length - 1),
    z: -gap * (medias.length -1)
})
```

Why is this placement managed in JavaScript? Because it allows for a dynamic result. Regardless of the number of cards in the DOM, the animation will work.

A timeline per card

Each card will start from the back of the deck. Then, it will bounce while getting closer to the screen as many times as there are cards. Finally, it will disappear by leaving the screen from the top. I define this series of movements by creating a `gsap.timeline()` for each card. This method allows linking multiple tweens and assigning them a series of default properties.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

ot play at the same time

Made With Gsap , since all the cards would start at the same position and move simultaneously.

To know when to start each timeline for each card, I calculate the total distance to scroll to cover the effect and divide it by the number of cards.

```
const distPerMedia = (pinHeight.clientHeight - window.innerHeight) / medias.leng
```

Notice the subtraction of `window.innerHeight` in this calculation. Indeed, when the `pin-height` division reaches the top of the viewport, the effect hasn't started yet, but it is already visible at the height of one screen.

Now I create the `gsap.timeline()` for each image. To loop through all the images, I use the native `forEach()` method. We also notice the use of the `scrollTrigger` object in the timeline. Its properties will specify when each timeline starts, thanks to the `distPerMedia` value calculated just before.

```
medias.forEach((media, index) => {
    const tl = gsap.timeline({
        scrollTrigger: {
            trigger: pinHeight, // Listen to pin-height position
            start: 'top top+=' + (distPerMedia * index),
            end: 'bottom bottom+=' + (distPerMedia * index),
            scrub: 0.5 // Progresses with the scroll and takes 0.5s to reach the scroll
        }
    })
})
```

Let's break down the code above: each timeline starts when the top of the `pin-height` division reaches the top of the viewport plus the value of `distPerMedia`. We progressively shift the `start` property depending on the `index` being processed. We apply the same logic for the `end` attribute.

In this way, each timeline will start more or less when the effect reaches the viewport, and the first card will be at the top of the stack, ready to disappear.

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS
Made With Gsap

orrespond to the movement of the cards in the deck. A card moves up as many times as there are cards in the deck. Notice the use of `"+="`, which directly adds a value relative to the current one.

```
medias.forEach((media, index) => {
    ...

    for(let i = 0; i < medias.length - 1; i++){
        tl.to(media, {
            // Gains one position in the deck queue
            y: "-=" + gap,
            z: "+=" + gap,
            ease:'back.inOut(3)' // Light bounce at the start and end of the animation
        })
    }
})
```

Each timeline ends with the tween that makes the card disappear from the screen. Notice the use of the `Math.random()` method for the `rotation` property. Just a reminder, `Math.random() - 0.5` returns a value between -0.5 and 0.5. When multiplied by the next value, we increase the amplitude of the deformation. In our case, each card will finish its movement with an angle between -25 and 25 degrees.

```
medias.forEach((media, index) => {
    ...

    tl.to(media, {
        yPercent: -80, // Moves up by 80% of its height
        y: '-50vh', // Moves up to half the screen
        scale: 1.2, // The card slightly enlarges as it disappears
        rotation: (Math.random() - 0.5) * 50, // Will be different for each media
        ease: 'power4.in' // Starts gradually
    })
})
```

Go further

For a different result, it could be interesting to use another ease for the movements in the deck. For example, using `ease: 'power1.inOut'` would smooth out the movements.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

ihance the fluidity and sliding

is well, visit our Go Further page to learn how to easily add it.

Final code

HTML CSS JAVASCRIPT

```
<section class="mwg_effect042">
    <div class="pin-height">
        <div class="container">
            <div class="text">
                <p>Dive into our fantastic world</p>
                <p>3D landscapes by Mo Eid</p>
            </div>

            <div class="medias">
                <img src="assets/medias/1.png" alt="" class="media">
                <img src="assets/medias/2.png" alt="" class="media">
                <img src="assets/medias/3.png" alt="" class="media">
                <img src="assets/medias/4.png" alt="" class="media">
                <img src="assets/medias/5.png" alt="" class="media">
                <img src="assets/medias/6.png" alt="" class="media">
            </div>
        </div>
    </div>
</section>
```

Wanna learn something else?

BACK TO THE COLLECTION

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

Never miss what's next

Your email SUBMIT

By submitting your email, you'll be the first to know about upcoming updates for Made With Gsap. You can unsubscribe at any time.

SOCIAL

X(TWITTER)
INSTAGRAM
BLUESKY
LINKEDIN

PAGES

HOME
COLLECTION
FAQ
PRICING

CONTACT

REACH US
SUPPORT

Made With Gsap

2025©

PERMISSIONS & TERMS

PRIVACY POLICY